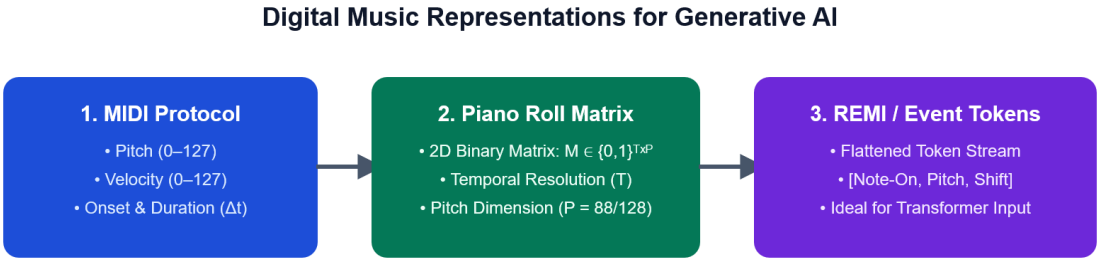
UNIT-IV: Music Generation with Generative AI

1. Introduction to Music Generation & Music Representation

Music generation models capture temporal dynamics, harmonic structures, and polyphonic dependencies across time. Unlike raw audio waveforms (continuous signals sampled at high rates like 44.1 kHz), structured digital representations encode musical events as discrete sequences.

Visual Representation Pipeline



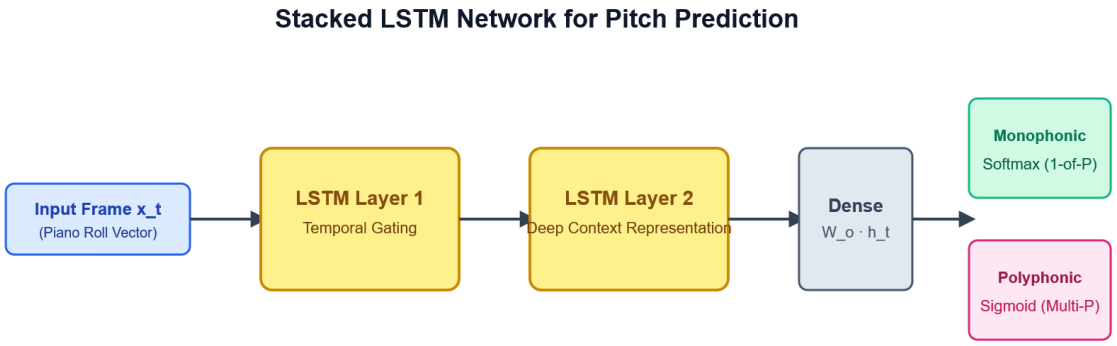
Key Digital Representations

- **MIDI (Musical Instrument Digital Interface):** A symbolic protocol that records performance events rather than raw sound. Key parameters include:
  - **Pitch:** Note frequency represented as integers (0 to 127, where $60 =$ Middle C).
  - **Velocity:** Key strike intensity/volume (0 to 127).
  - **Onset & Duration:** Precise start time (t_{start}) and hold time (Δt).
  - **Channel/Instrument:** Program assignments (e.g., piano, drums).
- **Piano Roll Representation:** A 2D binary matrix $M \in \{0,1\}^{T \times P}$, where T represents discretized time steps (e.g., 16th notes) and P represents the pitch dimension (typically 88 or 128 pitches). An entry $M_{t,p} = 1$ indicates that pitch p is active at time t.
- **Monophonic vs. Polyphonic Music:**
  - **Monophonic:** Exactly one note plays at any time step t (single melody line).
  - **Polyphonic:** Multiple notes can play simultaneously at time step t (chords, multi-instrument arrangements).

2. LSTM-Based Music Generation

LSTMs process music as temporal sequences by maintaining a persistent cell state C_t to learn long-term rhythmic patterns, pitch transitions, and chord progressions.

LSTM Architecture for Polyphonic Music Generation



Mathematical Formulation

Given an input vector x_t and previous hidden state h_{t-1}:

1. **Gating Functions:**

$$f_t = \sigma\big(W_f \cdot [h_{t-1}, x_t] + b_f\big) \quad \text{(Forget Gate)}$$
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{(Input Gate)}$$
$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad \text{(Candidate Memory)}$$

2. **Cell & Hidden State Update:**

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$
$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \text{(Output Gate)}$$
$$h_t = o_t \odot \tanh(C_t)$$

3. **Output Layer Strategy:**
   - **Monophonic Generation:** Applies Softmax over P pitch categories:

$$P(x_t = p \mid x_{<t}) = \frac{\exp\big(z_p\big)}{\sum_{j=1}^{P} \exp\big(z_j\big)}$$

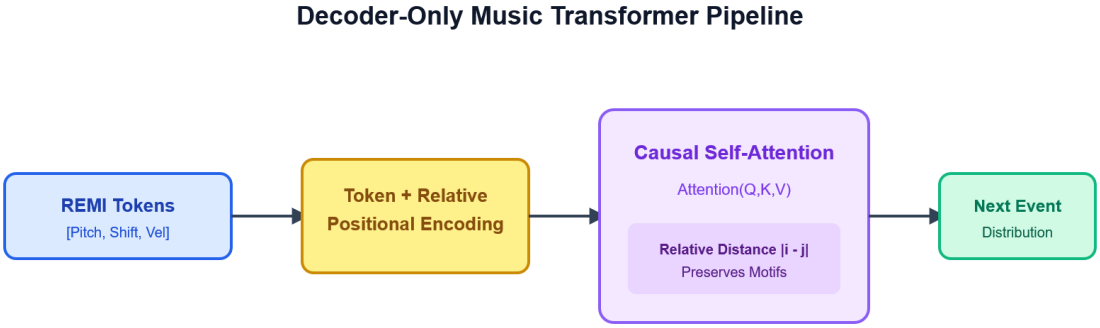
   - **Polyphonic Generation:** Applies independent Sigmoid activation for each pitch p to allow multiple simultaneous active notes:

$$P\big(M_{t,p} = 1 \mid M_{<t}\big) = \sigma\big(z_p\big) = \frac{1}{1 + e^{-z_p}}$$

3. Transformer-Based Music Generation

Transformer architectures replace recurrent connections with **Self-Attention Mechanisms**, processing long sequences in parallel to capture multi-bar themes, phrase repetitions, and harmonic structures.

Causal Transformer Architecture for Music



Key Components

- **Event-Based Tokenization (REMI / MIDI-Like Encodings):** Converts MIDI files into a flattened sequence of structural tokens rather than dense matrices. Tokens represent specific events:

  $\text{Tokens} = [\text{Note-On}(60), \text{Velocity}(80), \text{Time-Shift}(250\text{ms}), \text{Note-Off}(60)]$

- **Causal Self-Attention:** Computes attention weights while masking future timesteps to maintain an autoregressive generation pipeline:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

Where $M_{ij} = 0$ for $i \geq j$ and $M_{ij} = -\infty$ for $i < j$.

- **Relative Positional Encodings:** Standard absolute positional encodings struggle with musical concepts like key transpositions or repeating rhythm motifs. Relative

positional encodings inject structural relationships based on distance $|i - j|$ between notes, preserving musical motifs regardless of where they occur in a piece.

4. Evaluation, Fine-Tuning & Music Composition Applications

Evaluation Metrics for Music Generation

Evaluating generated music requires assessing both technical coherence and musical diversity.

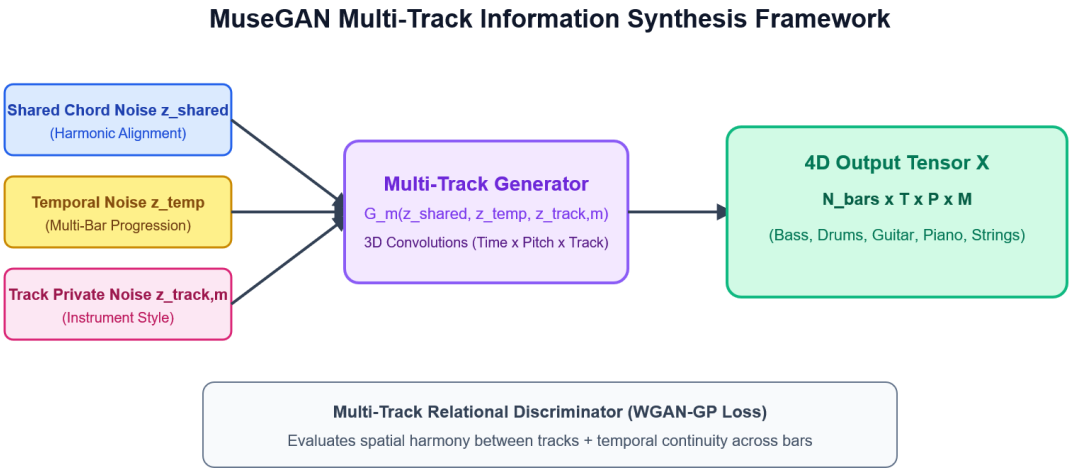
Evaluation Metric	Target Property	Description / Method
Pitch Class Histogram Distance	Harmonic Alignment	Measures KL divergence between the distribution of 12 chromatic pitch classes in generated music vs. a target dataset.
Scale Consistency	Musical Key Coherence	Calculates the ratio of generated notes that fit within a dominant musical scale (e.g., C Major).
Empty Bar Rate (EBR)	Structural Continuity	Measures the percentage of generated bars that contain no active note events.
Polyphonic Density	Texture Complexity	Calculates the average number of simultaneous active notes per bar or beat.
Perplexity (PPL)	Model Confidence	Measures how well the autoregressive model predicts unseen test MIDI event sequences.

Fine-Tuning & Composition Applications



5. MuseGAN: Full Architecture & Multi-Track Generation

MuseGAN is a foundational deep learning framework specifically created for multi-track polyphonic music generation using Generative Adversarial Networks (GANs).



A. Overview of MuseGAN Architecture

MuseGAN represents musical scores as 4D tensors and uses 3D Convolutional Neural Networks (3D CNNs) to handle time, pitch, and tracks jointly.

System Representation

$$\mathbf{X} \in \mathbb{R}^{N_{\text{bars}} \times T \times P \times M}$$

Where:

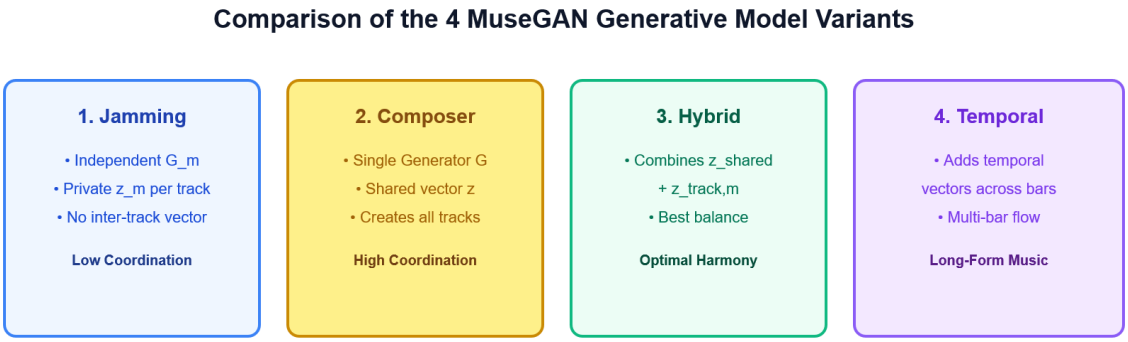
- N_{bars}: Number of musical bars (e.g., 4 or 8 bars).
- T: Temporal resolution per bar (e.g., $T = 96$ time steps per bar for 16th note resolution).
- P: Pitch dimension (typically $P = 84$ or 128 chromatic pitches).
- M: Number of tracks/instruments (e.g., $M = 5$: Bass, Drums, Guitar, Piano, Strings).

B. Structural Decomposition & Noise Vectors

To solve the coordination problem in multi-track generation (e.g., preventing instruments from playing in completely different keys), MuseGAN splits input noise into four specialized vectors:

1. **Inter-track Latent Variable (z_{shared}):** A shared noise vector fed to all instrument generators to control global chord structures and ensure all instruments remain in the same key.
2. **Intra-track Latent Variable ($z_{\text{track},m}$):** A track-private noise vector supplied only to instrument m to generate unique melodic and rhythmic traits for that specific instrument.
3. **Inter-bar Latent Variable ($z_{\text{temp, shared}}$):** Controls global temporal progression across multiple bars.
4. **Intra-bar Latent Variable ($z_{\text{temp, track},m}$):** Controls bar-to-bar stylistic variations for individual instrument tracks.

C. The 4 MuseGAN Model Variants



1. **Jamming Model:** Multiple independent generators $G_1, G_2, \ldots, G_M$ generate individual tracks independently using private noise vectors z_m. Tracks are combined without direct inter-track coordination.
2. **Composer Model:** A single unified generator G receives a shared noise vector z and creates all M tracks together, ensuring high harmonic alignment across instruments.
3. **Hybrid Model:** Combines shared inter-track noise z_{shared} with track-private noise $z_{\text{track},m}$. Each track generator receives both inputs:
$$\mathbf{X}_m = G_m\left(z_{\text{shared}}, z_{\text{track},m}\right)$$
4. **Temporal Model:** Extends the Hybrid model by adding temporal latent variables $z_{\text{temp},t}$ to ensure multi-bar rhythmic and thematic continuity across time.

D. Training MuseGAN on Polyphonic Music Datasets

1. **Dataset Preparation:** MuseGAN is typically trained on polyphonic datasets like the **Lakh MIDI Dataset (LMD)** or **Symbolic Music Datasets** converted into multi-track piano-roll tensors.
2. **Binary Thresholding & Binarization:** Because continuous outputs from generator layers ([0,1] via Sigmoid) do not form discrete binary notes, a thresholding strategy or deterministic binary activation function is applied during inference.
3. **WGAN-GP Optimization:** To handle training instability in multi-track GANs, MuseGAN uses **Wasserstein GAN with Gradient Penalty (WGAN-GP)** loss:
$$\mathcal{L}_D = \mathbb{E}_{\tilde{\mathbf{X}}\sim P_g}\left[D\left(\tilde{\mathbf{X}}\right)\right] - \mathbb{E}_{\mathbf{X}\sim P_r}[D(\mathbf{X})] + \lambda \mathbb{E}_{\hat{\mathbf{X}}\sim P_{\hat{\mathbf{X}}}}\left[\left(||\nabla_{\hat{\mathbf{X}}} D\left(\hat{\mathbf{X}}\right)||_2 - 1\right)^2\right]$$

E. Generating Complex Music Compositions with MuseGAN

- **Multi-Bar Generation:** Uses 3D Transposed Convolutions to expand temporal dimensions from a single bar vector to multi-bar compositions (e.g., 4 bars of 16 steps each).
- **Track Interdependence:** By tuning the ratio between z_{shared} and $z_{\text{track},m}$, composers can control whether generated instruments strictly follow a shared chord progression or improvise independently.
- **Human-in-the-Loop Interaction:** Composers can fix specific instrument tracks (e.g., lock a human-written piano chord sequence) and use MuseGAN's discriminator/generator to synthesize complementary bass, guitar, and drum tracks around the fixed sequence.